MATHEMATICAL PROOF-LANGUAGE DESIGN

# Bryony

## Contract-Indexed Mathematical Reasoning with Proof-Carrying Requirements

---

**The proposal.** Let an author state the mathematical construction and the next meaningful step. Infer a finite family of sufficient premises for that step, retaining a proof recipe for every alternative. Discharge those premises without changing the statement, the mathematical objects, or the chosen structures. Export ordinary Lean proofs.

This fourth-round design develops the property-aware architecture of the two round-three syntheses into an explicit requirement calculus, a bounded grounding protocol, and a small executable reference model. It examines native type-system extensions as a separate engineering choice, rather than confusing stronger author-facing typing with stronger kernel conversion.

Prepared for Vladimir Reshetnikov

ChatGPT | 6 September 2026

Evidence boundary. The accompanying Python model was executed: 23 test methods passed, including differential checks on 400 finite rule systems. Soundness and relative completeness of the finite calculus are proved on paper. The generated Lean proof templates were not compiled in this preparation environment. This is not a completed Lean frontend or a measured author-productivity result.

**Abstract**

A useful mathematical proof language should compress routine connections, not the distinctions that make a statement true. The round-three proposals largely settle the representation of those connections: properties of fixed terms, relational contracts on fixed operations, theorem-backed inference, and ordinary Lean evidence. Bryony takes the remaining implementation questions as its starting point. Its central object is a *proof-carrying requirement family*: an antichain of sufficient prospective premise sets for an exact demanded proposition, with a proof template attached to each set. This supports explanation, conditional elaboration, synthesis, and repair without treating missing premises as facts.

We specify a finite fragment with soundness, termination, and relative completeness; describe how actual Lean expressions can be grounded into that fragment without changing meaning; and separate witness construction from property propagation. Worked examples cover weighted subgraph Fubini transport in ProveIt, exact Frey coefficients and tensor-algebra interfaces in the Fermat repository, and behavior-aware synthesis using Leant. Almost-everywhere equality, exact division, behavioral variance, dependent witnesses, certificate reconstruction, and context transport receive explicit interfaces. A native refinement former is considered with its translation obligations, but is not required for the first implementation. An executable propositional reference engine and adversarial tests accompany the article. The proposed experiment isolates property-aware authoring from the benefits of new libraries, automation, and rendering.

## Contents

# 1 The design decision

## 1.1 Compress connections, preserve decisions

A mathematician ordinarily writes "the integrand is integrable, so exchange the integrals," not a sequence of instructions about indicator functions, measure restrictions, coercions, and the orientation of an equality. The omitted information is not all of one kind. Some is reconstructible from definitions; some is an application of an established lemma; some is a genuinely additional assumption; and some is a choice of what the argument is about. A language succeeds only if it distinguishes these cases.

Bryony gives the author responsibility for *semantic checkpoints*: the objects being constructed, the relation being claimed, the important reduction, and any method that is part of the intended argument. Between checkpoints, a property-aware elaborator assembles routine evidence. At a blocked checkpoint it displays mathematical requirements, not merely a failed tactic invocation. At no checkpoint may the system silently weaken the conclusion, choose a more convenient measure, or install an extra assumption.

The key interface is not a command called "obvious." It is a request of the form

$$\text{establish } G \text{ about these fixed objects, under this context and policy.}$$

The answer may be a proof, or several conditional proof recipes. For example, a denominator can be shown nonzero from positivity, negativity, a unit witness, or a direct nonzero hypothesis. These are different sufficient routes. There need not be a useful single weakest condition, and a failed route does not refute the goal.

## 1.2 What is inherited, and what changes

The two round-three syntheses agree on the main architecture: retain ordinary Lean objects; attach scoped proofs of their properties; make contracts active at use sites; distinguish structures from properties; and keep unrestricted semantic entailment out of kernel conversion. They also identify what has not been tested: a real inference interface, grounding, rewriting of anchors, incremental context reconstruction, and authoring benefit against an equally equipped Lean baseline [1, 2].

Bryony adopts that architecture rather than proposing a tenth incompatible spelling of a refinement row. Its additional commitments are specific:

1. A demand returns a *family of proof-carrying sufficient requirements*, with a precise finite interpretation and an executable model.
2. Grounding and proof search have separate budgets and separate completeness claims. Meaning-bearing expressions are fixed before ground closure begins.
3. A proof step has an identity that includes its actual candidate, structures, relation, and method policy. Support minimization never merges different witnesses or different meanings.
4. The same conditional proof templates support explanation and repair. They do not become unconditional evidence until their premises are supplied.

This is an implementation-oriented refinement of the campaign, not a claim to have invented refinement types, assumption-based reasoning, or proof-producing automation.

### 1.3 The first product is a Lean extension

The initial deliverable should be a Lean package with property-implicit arguments, a small registry, a requirement view, and ordinary Lean export. A separate file syntax is optional. A mathematical document renderer may consume the same records. A native kernel extension is a distinct experiment, discussed in Section 13.

This choice still extends the type system *experienced by the author*. A function can be checked against a behavioral specification, an argument can be required to satisfy a property, and a partial construction can retain explicit obligations. The fact that these judgments elaborate to dependent functions and subtypes does not make the author-facing extension merely typography. Its substance is the checking discipline and the evidence it inserts.

## 2 Reading the corpus without overstating the evidence

### 2.1 Snapshots and source selection

Both round-three synthesis sources were examined at Brainstorm commit `bf3333905995060870f8585e297ffc16f3fe7e86`, including their reconciliation of disagreements. The relevant source studies here use ProveIt commit `24ce8bd7` and the Fermat repository commit `aa2d8b34`, with complete identities in the bibliography. ProveIt's toolchain file pins Lean 4.32.0. The selected files are development examples, not held-out evaluation tasks [1, 2, 3, 4, 5, 6].

The analytical study reads both weighted section-transport theorems in `SubgraphFubini.lean`. The arithmetic study reads the coefficient-map argument and its surrounding invariant proofs in `S_FreyPackage_freyCurveInt_map.lean`. The structural study reads the finite tensor-family construction in `S_Algebra_exists_algHom_equiv_pi.lean`. These are not a build or an audit of either full repository. The Fermat repository's own verification claims are not needed to assess these local interfaces.

There is a provenance discrepancy worth preserving. The syntheses discuss a later Leant snapshot with prefix `823259f7`. That short reference could not be resolved through the public GitHub endpoint during this review. The publicly accessible `main` resolved instead to `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. The present Leant analysis uses that accessible source, particularly `Verification.hs`, together with the repository documentation. Claims about the later local snapshot remain attributed to the syntheses; they are not presented as freshly corroborated source facts [7, 8, 1, 2].

### 2.2 The two syntheses are complementary, not interchangeable

*From Properties to Proof Obligations* is especially useful on alternative sufficient contracts, satisfiable helper contexts, obligation frontiers, and the four separate guarantees behind finite evidence. *Nine Refinements* contributes a finer commitment matrix, a larger mutation catalogue, source-level implementation questions, and a concrete account of the previously supplied Lean specimens. Its later reconciliation corrects overstatements earlier in the same source; those corrections govern this proposal [1, 2].

Three corrections directly affect Bryony. First, a sound checker can prove every claim it accepts without being complete. Second, unrestricted affine coefficient comparison is not complete for source-level list-length equality over an empty element type. Third, the tensor-family example needs finite and free *factors*, not merely a finite indexing type. A language intended to prevent missing premises must preserve these distinctions in its own examples.

The syntheses report successful compilation of seven small Lean cores containing 49 named theorems,

with different scopes of axiom inspection. Those are useful inherited artifacts. They do not constitute an implementation of a property elaborator, and they were not recompiled here. Bryony's fresh executable evidence is a different, explicitly limited artifact: the finite propositional engine of Section 15.

### 2.3 Where the source verbosity comes from

The files expose at least four costs. *Mathematical decomposition* supplies the argument: introduce a subgraph integrand, prove exact divisibility, or construct a tensor product. *Interface assembly* connects that argument to existing theorem statements. *Representation transport* moves among indicators, restricted measures, integer quotients, rational coefficients, and equivalence compositions. *Environment management* fixes instances, namespaces, and generated infrastructure.

Only some of these should be hidden by property inference. A shorter generated preamble does not demonstrate a better mathematical type system. Conversely, an integrability bridge can be valuable even when its source-line saving is small, because it exposes why Fubini is licensed. The evaluation must classify the work removed instead of adding all deleted lines into one compression ratio.

## 3 The author-facing language

### 3.1 Objects, knowledge, and constructions

The following notation is a proposed mathematical surface, not the parser implemented by the companion:

```
fix x : Real where 0 < x
know x != 0

construct q : Int satisfying 16 * q = numerator
show (q : Rat) = (numerator : Rat) / 16
  by exact_quotient_transport
```

The first line introduces an object and a hypothesis. The second demands a consequence, which must be proved. The third authorizes a new witness under a fixed specification. The last demands a particular equality. These commands must not be synonyms.

A local annotation such as

$$x : A \text{ where } P(x), Q(x)$$

means that the same term $x$ has the same type $A$, and proofs of the displayed predicates are available. The local environment retains $x$ and those proofs separately. An exported value may instead use $\{x : A \mid P(x) \wedge Q(x)\}$ or an ordinary structure. Local inference should not repeatedly rebind $x$ to nested subtypes merely to add knowledge.

The predicate need not be unary in substance. An inverse-pair property mentions two functions; exactness mentions several maps; naturality mentions a family and its action on morphisms. A row is a presentation of propositions, not a prohibition on relations among several objects.

### 3.2 Property-implicit parameters

A proposed declaration may mark selected proof parameters as requirements:

```
method cast_exact_quotient (n d q : Int)
  requires balance : d * q = n
  requires denominator : (d : Rat) != 0
  ensures (q : Rat) = (n : Rat) / (d : Rat)
```

Its mathematical content is an ordinary Lean theorem with ordinary proof arguments. The new feature is that applications generate demands for the marked premises and offer registered routes to them. A user can always supply a proof explicitly. The attribute does not turn an arbitrary proposition into a global typeclass, nor does it change the meaning of Lean's existing division operators.

The registry validates the declaration against the actual theorem type. A claimed "balance" role cannot point at an unrelated binder. Implicit type parameters and structure dictionaries are elaborated first. Roles are conveniences for matching and diagnostics, not axioms about what the theorem means.

## 3.3 Three forms of function interface

For a total function $f : A \to B$, a behavioral annotation denotes

$$\mathsf{Beh}(f; P, Q) := \forall x : A,\ P(x) \to Q(x, f(x)), \tag{1}$$

where $Q : A \to B \to \mathsf{Prop}$. This differs from a function on the refined domain,

$$f : \{x : A \mid P(x)\} \to B,$$

and from a witness-producing dependent function,

$$f : \prod_{x:A} \prod_{h:P(x)} \{y : B(x) \mid Q(x, h, y)\}.$$

The first has a value even where its guard fails; only the theorem is conditional. The second requires a domain witness before application. The third can make the result depend on an earlier witness and its specification. The surface must make this distinction recoverable.

A synthesis request can use a refined function space:

```
construct sort : List A -> List A satisfying
  forall xs, Permutation (sort xs) xs
          and Sorted order (sort xs)
```

Here the author delegates the program, not the specification or the order. Stability is an additional clause when wanted; length preservation is not a substitute for permutation. Whole-function properties such as monotonicity or noninterference can likewise appear as predicates on the function, rather than being forced into a pointwise postcondition.

## 3.4 What the compact document displays

A closed checkpoint shows its claim and the main reason. Expansion reveals the actual theorem applications, inferred premises, chosen structures, and proof. An open checkpoint shows the unresolved premises by default. A conditional result is displayed as conditional, never with the same status as a completed theorem.

Every displayed mathematical assertion receives its own check, even when the final theorem does not use it. Otherwise a document could contain an unsupported intermediate statement while its terminal theorem remained valid. Unused explanations can remain ordinary prose, but any sentence presented as a proved assertion must correspond to evidence.

# 4 A conservative semantic core

## 4.1 The authoritative context

Let $\Gamma$ be a dependent Lean context, including universes, local definitions, structure arguments, and hypotheses. A property index $K$ stores propositions with accepted proof terms in $\Gamma$. It is an index into the logical context, not an independent source of truth.

For an already elaborated $e : A$, write

$$\Gamma; K \vdash e : A \triangleright P$$

when $\Gamma \vdash p : P(e)$ for a retained proof $p$. Conjunction combines evidence about the same $e$. A theorem $P(e) \to Q(e)$ produces a new proof of $Q(e)$. Equality $h : e = e'$ transports through the actual dependent predicate. None of these rules changes kernel conversion.

A blocked check has a different judgment:

$$\Gamma; K \vdash e : A \triangleright P \mathbin{!} \{(S_i, t_i)\}_{i \in I}, \qquad \Gamma \vdash t_i : \Big(\bigwedge S_i\Big) \to P(e). \tag{2}$$

The prospective propositions in $S_i$ are well-typed in $\Gamma$ but are not asserted. The conjunction is shorthand for an ordered sequence of proof parameters in the nondependent fragment. The actual dependent representation is a telescope. This judgment records conditional proof templates; it does not allow an unchecked term into the exported environment.

## 4.2 Discharging a requirement

If every member of $S_i$ has an actual proof in $\Gamma$, applying $t_i$ yields evidence for $P(e)$. This is the only automatic promotion from a prospective route to established knowledge. An empty support gives a closed proof relative to the already accepted context. No route at all means that this registry and this search produced no recipe. It does not mean $\neg P(e)$.

This separation permits useful partial work. One can construct a conditional adapter before knowing which proof of divisibility will be used. It also prevents a common circularity: the conditional conclusion may not be placed in $K$ and then reused to prove the condition on which it depends.

## 4.3 Behavioral consequence and composition

The variance of contracts follows directly from (1). If

$$P'(x) \Rightarrow P(x), \qquad P'(x) \wedge Q(x, y) \Rightarrow Q'(x, y),$$

then $\mathsf{Beh}(f; P, Q)$ yields $\mathsf{Beh}(f; P', Q')$. A client can offer stronger assumptions and accept a weaker guarantee. For an implementation replacing another implementation, the offered precondition must be no stronger and the offered guarantee no weaker than the client's interface requires.

**Proposition 4.1** (Composition retains the intermediate)**.** *Suppose* $\mathsf{Beh}(f; P, Q)$ *and* $\mathsf{Beh}(g; U, V)$*, together with*

$$P(x) \wedge Q(x, y) \Rightarrow U(y), \qquad P(x) \wedge Q(x, y) \wedge V(y, z) \Rightarrow W(x, z).$$

*Then* $\mathsf{Beh}(g \circ f; P, W)$*.*

*Proof.* Fix $x$ and a proof of $P(x)$. Apply the first contract to get $Q(x, f(x))$. Apply the first bridge at that exact intermediate value to get $U(f(x))$. The second contract gives $V(f(x), g(f(x)))$, and the second bridge gives $W(x, g(f(x)))$. □

The codomain of $f$ matching the domain of $g$ is not the behavioral bridge. If a construction changes the witness, the next requirements are about the new witness. For dependent operations, replace the simple arrows by dependent functions and preserve the ordered bindings of the intermediate data and proofs.

### 4.4 What a translation theorem must cover

For a restricted source calculus, a preservation theorem should relate an authored judgment to the type of its translation, including translated contexts. Local refinement rules translate by theorem application; property-implicit arguments become explicit proof arguments; discharged conditional templates are applied to evidence; construction produces a dependent pair or an explicitly authorized choice.

Checking an arbitrary emitted theorem in Lean is not the same result. A frontend could emit a proof of the wrong statement. The source-to-target correspondence, treatment of implicit arguments, and classification of metavariables belong in the translation specification. The companion only implements the finite propositional part of this architecture; the full correspondence theorem is an implementation obligation, not a claimed accomplishment of its tests.

## 5 Proof-carrying requirement families

### 5.1 The finite problem

Fix a finite set $\mathcal{Q}$ of elaborated propositions, a subset $K \subseteq \mathcal{Q}$ of known facts, a subset $H \subseteq \mathcal{Q}$ of allowed *prospective premises*, and a finite set $\mathcal{R}$ of ground rules

$$r : P_1 \to \cdots \to P_m \to Q, \qquad P_j, Q \in \mathcal{Q}.$$

Each rule in the intended Lean implementation is an instantiated theorem, with its data parameters and structures fixed. A rule with no premises is a proved fact. Grounding may retain local real variables or functions: "ground" means no unresolved rule-instantiation variables, not that the mathematical domain is finite.

Let $\mathrm{Cl}_{\mathcal{R}}(X)$ be the least set containing $X$ and closed under the rules. For a demand $G \in \mathcal{Q}$, define

$$\mathcal{A}_G = \mathrm{Min}_{\subseteq}\{S \subseteq H : \ G \in \mathrm{Cl}_{\mathcal{R}}(K \cup S)\}. \tag{3}$$

These are inclusion-minimal sufficient supports *in the fixed rule system.* They need not be logically weakest assumptions in Lean, smallest by cardinality, cheapest to prove, mutually exclusive, or mathematically necessary.

The choice of $H$ is part of the request profile. It should normally contain recognizable boundary premises of the selected operations, such as divisibility or integrability, not every proposition reachable from arbitrary theorem search. Offering $G$ itself as a prospective premise is logically valid but usually an unhelpful explanation; the interface can exclude such routes from its preferred display without altering the meaning of accepted proofs.

### 5.2 An algebra of alternatives

For a family $A$ of finite supports, let $\mathrm{Min}(A)$ remove every support strictly containing another. Define

$$A \oplus B = \mathrm{Min}(A \cup B), \qquad A \otimes B = \mathrm{Min}\{S \cup T : S \in A, \ T \in B\}.$$

The additive zero is the empty family $\varnothing$, and the multiplicative unit is the singleton family $\{\varnothing\}$. Alternative proof routes use $\oplus$; the simultaneous premises of a rule use $\otimes$.

The meaning of a family is the upward-closed collection

$$\uparrow A = \{U \subseteq H : \exists S \in A,\ S \subseteq U\}.$$

This removes a possible ambiguity in the direction of inclusion: a sufficient set $S$ remains sufficient when more prospective premises are supplied. A support $\{a\}$ therefore dominates $\{a, b\}$ for the *same goal and the same policy.* No implication between $a$ and $b$ is being assumed.

Attach an immutable proof tree to every retained support. A known fact has support $\varnothing$. A prospective premise $h$ has support $\{h\}$ and proof template $h \to h$. A rule node unions the supports of its children and applies the corresponding theorem. Repeated occurrences of the same prospective proof do not create new logical assumptions; contraction is ordinary Lean reasoning.

## 5.3 The algorithm

Initialize the labels of known facts with empty support and those of prospective premises with their singleton supports. For each rule, combine one retained proof from each premise label. Insert the union support at the conclusion unless an existing support is a subset; when inserting, remove strict supersets. Repeat until no label changes.

```
for q in Known:
  insert(q, {}, KnownProof(q))
for h in Prospective:
  insert(h, {h}, HypothesisProof(h))
repeat:
  changed := false
  for rule (p1, ..., pm => q) in FrozenRules:
    for (s1,t1), ..., (sm,tm) in snapshot(Label[p1], ..., Label[pm]):
      s := union(s1, ..., sm)
      changed |= insert(q, s, Apply(rule, t1, ..., tm))
until not changed
```

Snapshotting the premise labels avoids mutating a collection while iterating over it. Proof nodes refer to already constructed immutable nodes, so even a cyclic rule graph produces only finite derivation trees. The reference implementation retains the first proof of a support; it does not optimize proof size or method cost.

**Theorem 5.1** (Conditional soundness)**.** *Every retained pair $(S, t)$ at a proposition $G$ represents a derivation of $G$ from $K \cup S$. With proofs for the known facts and theorem evidence for the rules, it translates to a Lean proof template $(\bigwedge S) \to G$ in the fixed context.*

*Proof.* Induct on proof-tree construction. A known leaf uses its retained proof. A prospective leaf uses the corresponding proof parameter. At a rule node, the union support supplies every parameter needed by every child. Apply the induction hypotheses and then the actual rule theorem. Removing alternative supports does not invalidate a retained tree or any older tree referenced by it. □

**Theorem 5.2** (Termination and relative completeness)**.** *The untruncated algorithm terminates. At termination, its label at $G$ is exactly $\mathcal{A}_G$ from (3), up to the choice of proof tree for an equal support.*

*Proof.* Whenever a support $S$ is inserted at $q$, no existing support is a subset of $S$. Thus $S$ was absent from the upward closure of the old label and is present afterward. Removing supersets leaves that upward closure unchanged. Each successful insertion strictly enlarges an upward-closed subset of the finite set $2^H$. Across all propositions there are at most $|\mathcal{Q}|2^{|H|}$ such insertion events. Every scan has finitely many rule combinations, so a full unchanged scan eventually occurs.

For completeness, fix $S \subseteq H$ such that $G \in \mathrm{Cl}_{\mathcal{R}}(K \cup S)$. Consider a finite forward derivation witnessing membership in this least closure. We show by induction on this derivation that at the final fixed point the label of each derived proposition contains some support contained in $S$. A known leaf has the empty support, or a support dominating it, necessarily also empty. A prospective leaf in $S$ starts with its singleton; if removed, it is replaced by a smaller support. At a rule application, the induction hypotheses supply final premise supports contained in $S$. Their union is contained in $S$ and was considered in the final scan; it is retained or dominated by a smaller conclusion support. This proves coverage of every sufficient $S$.

Soundness shows that every stored support is sufficient. Since stored labels are antichains, coverage forces their elements to be precisely the inclusion-minimal sufficient supports. □

### 5.4 Why the qualifications matter

The literal antichain need not grow monotonically: $\{a, b\}$ can disappear when $\{a\}$ is found. Its *upward-closed meaning* grows monotonically. This is the invariant behind the termination proof and the reason a naive argument counting only currently stored rows would be inadequate.

The bound is exponential in the number of prospective premises. Even a final antichain can contain $\binom{|H|}{\lfloor |H|/2 \rfloor}$ supports. It also excludes the cost of grounding, checking theorem instances, and building large proof terms. A practical system may restrict $H$, share proof DAGs, or show only a ranked subset. Truncating the internal support family sacrifices the completeness claim; truncating only its display does not. Either way, every retained proof remains subject to replay.

Cycles are harmless without seeds. Rules $P \to Q$ and $Q \to P$ do not derive either proposition from an empty starting set. With a prospective premise $P$, they can produce a conditional recipe from $P$, which is legitimate and visibly conditional. Bryony does not confuse this with an induction principle or a proof of $P$.

### 5.5 Prior art and the precise increment

Minimal supporting environments are a central idea of de Kleer's assumption-based truth-maintenance systems. Liquid Types provide an important precedent for making a finite logical vocabulary useful in type inference. Proof and data provenance have also long been organized by algebraic combinations of alternatives and dependencies [15, 16, 17]. Bryony does not claim these mechanisms as new.

The intended increment is their disciplined placement at a Lean authoring boundary: every ground rule must have theorem evidence; prospective premises are never default beliefs; an exact candidate and its dependent context are part of the request; and replay reconstructs a proof at the original target. In particular, this design does not automatically remove inconsistent mathematical contexts as "nogoods." Proof by contradiction remains valid. Method-restricted benchmarks can separately forbid a contradiction shortcut when they are intended to test a particular arithmetic or analytical route.

## 6 Grounding actual Lean expressions

### 6.1 Separate three classes of holes

Before property inference, the elaborator distinguishes *meaning holes*, *authorized construction holes*, and *proof holes*. Meaning holes determine the requested statement: its universes, objects, operations, measures, branches, quantifier order, and conclusion strength. They must be resolved

or reported as ambiguous before a proof provider runs. Authorized construction holes may be filled with data satisfying an explicitly fixed specification. Proof holes ask for evidence of an already fixed proposition.

Being proposition-typed is not, by itself, an adequate classification algorithm. A proof argument can occur in dependent data, and an unresolved data argument can be hidden inside an apparently ordinary proposition. The implementation must compute the dependency closure of the sealed target and its fixed parameters, not merely inspect each metavariable's printed type. An explicitly supplied construction specification authorizes a particular dependency boundary; it does not authorize arbitrary changes elsewhere in the target.

A first implementation should reject a provider attempt that assigns a frozen metavariable. Lean elaboration snapshots permit transactional search, but the invariant must be checked against the actual metavariable assignments and elaborated target, not inferred from which tactic was called. Re-elaborating accepted output against a saved goal is a second check, not permission to forget the first.

## 6.2 A finite generation protocol

The first Bryony profile uses a finite pool $\mathcal{T}$ of already elaborated terms: local variables and definitions, designated subterms of the target, and explicitly named intermediate objects. Each has fixed universe and structure arguments. A registry rule schema declares which data parameters are matchable, which are fixed by its conclusion, and which may be selected from $\mathcal{T}$. Proof parameters become premises.

For each demand, generation matches the conclusion of an allowed rule against the demand and enumerates only the remaining declared choices from $\mathcal{T}$. It retains a rule instance only after Lean checks its complete type and after all data parameters are determined. The resulting premise propositions are added to the finite vocabulary. Demand depth, total instances, matching work, and normalization work have independent budgets.

For a truly finite protocol, the vocabulary is not allowed to generate new terms indefinitely. A schema that would repeatedly replace $x$ by $f(x)$ cannot grow $\mathcal{T}$ inside a closure epoch. A construction may add an explicitly accepted new term and start a new epoch, with a new bound. The fixed-point theorem applies within each epoch. It says nothing about an unbounded succession of constructions.

A supported first-order matching fragment can enumerate all permitted instances over a fixed pool. General dependent unification may leave unresolved choices; these are unsupported or budget-limited generation cases, not grounds for claiming that no mathematical proof exists. The engine should report both the finite registry it actually closed and whether the requested generation profile completed.

## 6.3 What is checked at registration

A registration stores the theorem constant and universe instantiation policy, the elaborated binder roles, the conclusion pattern, the fixed structure dependencies, and the permitted proof method profile. Registration succeeds only after checking that every premise role is a genuine premise of the theorem and every advertised conclusion is its actual conclusion after instantiation.

A property rule should not hide fresh data construction in a proof-search side effect. A theorem asserting existence belongs to a construction service unless the demand is itself existential. Likewise, extracting a bundled structure's multiplication changes the selected data only when that bundle was already the specified structure source. A convenient dictionary is not evidence that it is the intended dictionary.

### 6.4 Relative completeness, stated in two layers

The ground engine is complete for its fixed $\mathcal{Q}, K, H, \mathcal{R}$. A generation profile can additionally be complete for its own finite term pool and supported matching grammar. Neither entails completeness for Lean proofs, all possible theorem instances, all mathematical constructions, or all sufficient conditions. These are separate claims with separate failure states.

This two-layer contract answers an important round-three gap. "Bounded inference" is not a single property of a monolithic elaborator. A user needs to know whether a finite closure completed, whether instance generation was cut short, and whether a requested route lies outside the supported profile. Only the first question is settled by Theorem 5.2.

## 7 Worked arithmetic: exact Frey coefficients

### 7.1 What the Lean file actually does

The selected Fermat source proves that an integral Weierstrass model maps to the intended rational model. It applies coefficient extensionality and then justifies casting integer quotients into rational quotients. The nontrivial coefficients have numerators

$$n_2 = b^p - 1 - a^p, \qquad n_4 = -a^p b^p,$$

with divisors 4 and 16. The source explicitly derives evenness, power divisibility, and the residue of an odd power modulo four before applying the cast lemma [5]. The apparent clutter is partly the manual connection between those properties and the next operation.

A useful helper context is smaller than the surrounding counterexample package:

$$a, b \in \mathbb{Z}, \quad p \in \mathbb{N}, \quad 4 \le p, \quad p \text{ odd}, \quad a \equiv 3 \pmod 4, \quad 2 \mid b. \tag{4}$$

This context is inhabited, for example by $(3, 2, 5)$. It does not include a Fermat equation or a premise eventually used to derive a contradiction. The two syntheses emphasize this separation for an informative authoring experiment [1, 2].

**Proposition 7.1** (Operation-specific divisibility)**.** *Under* (4)*,* $4 \mid n_2$ *and* $16 \mid n_4$*. The second conclusion needs only* $2 \mid b$ *and* $4 \le p$*, with no condition on* $a$*.*

*Proof.* Write $b = 2k$. Since $p \ge 4$, $b^p = 16 \cdot 2^{p-4} k^p$, so $16 \mid b^p$. This implies $4 \mid b^p$ and $16 \mid -a^p b^p$. Modulo four, $a \equiv -1$ and oddness gives $a^p \equiv -1$. Hence $n_2 \equiv 0 - 1 - (-1) = 0$. □

The exponent assumptions are sufficient for this helper, not a characterization of all triples for which the quotients are exact. For $n_4$, divisibility of $a^p$ could provide another route. Bryony must not mistake a selected helper's premises for mathematically necessary conditions.

### 7.2 Separate division, transport, and inversion

For integers $d, n, q$, a balance equation $dq = n$ survives every ring homomorphism $j : \mathbb{Z} \to R$:

$$j(d)j(q) = j(n).$$

To rewrite this as $j(q) = j(n)/j(d)$ in a field, one also needs $j(d) \neq 0$. In a general ring, an inverse-based formula needs a unit witness. Nonzero, cancellable, and invertible are not interchangeable predicates. In particular, a nonzero integer may map to zero in positive characteristic.

Imported integer division keeps its original totalized semantics. Bryony's `exact_quotient` is an explicit interface: either it constructs $q$ with $dq = n$, or it proves that an already specified quotient

expression has that balance property. It may not reinterpret every slash in the source as exact division.

At $(a, b, p) = (3, 2, 5)$ the quotients are $q_2 = -53$ and $q_4 = -486$. Removing one premise at a time gives useful failures: $(3, 2, 4)$ breaks the odd-power residue route, $(3, 3, 5)$ breaks evenness, $(3, 2, 3)$ breaks the bound needed for the second numerator, and $(1, 2, 5)$ breaks the residue assumption. These are counterexamples to particular weakened helper statements, not to the original theorem. The companion checks these four neighbors and 288 positive triples using exact integer arithmetic.

### 7.3 What the requirement family contributes

For the fixed second numerator and quotient, suppose the registry offers the route

$$(2 \mid b) \wedge (4 \leq p) \Rightarrow 16 \mid b^p \Rightarrow 16 \mid n_4 \Rightarrow \text{the requested cast identity},$$

with the rational denominator fact already proved. It also permits the direct prospective premise $16 \mid n_4$. The demand has two minimal supports:

$$\{\{16 \mid n_4\},\ \{2 \mid b,\ 4 \leq p\}\}. \tag{5}$$

These are alternative proof interfaces, not two possible values of the quotient. The companion's `frey4.bry` models exactly this propositional shape. It produces the two supports with proof trees, after two scans and eight insertions.

The practical display is mathematical:

```
To transport this integer quotient to Rat, establish either:
  (1) 16 divides the displayed numerator; or
  (2) b is even and 4 <= p.
The destination denominator has already been proved nonzero.
```

When the available context supplies the second route, it closes without a new assumption. When a later edit removes evenness, the direct divisibility route remains visible. The editor should not silently reinsert evenness, nor silently replace the quotient by another object.

A selected proof route may use fewer facts than the public helper interface. Bryony therefore records three different sets: the interface premises, the premises supplied by this application, and the dependencies of the retained proof. None of these sets, by itself, establishes that a hypothesis is mathematically indispensable.

## 8 Worked analysis: Fubini with the right measures and fibers

### 8.1 The statement worth preserving

ProveIt's `SubgraphFubini.lean` proves two weighted Bochner-integral transport formulas under arbitrary s-finite measures. Its scalar field can be real or complex, and its target is a complete compatible normed space. Let $\mu$ be the measure on $\Omega$, $\nu$ a measure on $\mathbb{R}$, $Q : \Omega \to \mathbb{R}$ measurable, $w$ integrable with respect to $\mu$, and $k$ integrable on $s$ with respect to $\nu$. With the scalar and completeness structures fixed as in the source, the strict-subgraph formula is

$$\int_\Omega w(\omega) \bullet \left( \int_{(-\infty, Q(\omega)) \cap s} k(t)\, d\nu(t) \right) d\mu(\omega) = \int_s \left( \int_{\{\omega : t < Q(\omega)\}} w(\omega)\, d\mu(\omega) \right) \bullet k(t)\, d\nu(t). \tag{6}$$

The source does not require positivity, probability measures, a density, or measurability of $s$ as an additional hypothesis [4]. A proposed shorthand that inserts these assumptions would shorten the proof by changing its generality.

## 8.2 The mathematical checkpoint sequence

The author's essential construction is the joint integrand. Set $\nu_s = \nu|_s$ and

$$A = \{(\omega, t) : t < Q(\omega)\}, \qquad H(\omega, t) = \mathbf{1}_A(\omega, t)\,(w(\omega) \bullet k(t)).$$

Measurability of $Q$ gives measurability of $A$. Product integrability of the scalar weight and vector kernel gives integrability of $(\omega, t) \mapsto w(\omega) \bullet k(t)$ under $\mu \times \nu_s$. Restriction by the measurable indicator preserves integrability. Fubini is now licensed for $H$.

The remaining work identifies the two sections:

$$\int H(\omega, t)\, d\nu_s(t) = w(\omega) \bullet \int_{(-\infty, Q(\omega)) \cap s} k(t)\, d\nu(t),$$
$$\int H(\omega, t)\, d\mu(\omega) = \left( \int_{\{\omega : t < Q(\omega)\}} w(\omega)\, d\mu(\omega) \right) \bullet k(t).$$

The Lean proof builds these identities explicitly using indicator integrals, scalar extraction, and the equality between iterated measure restriction and intersection restriction. The final calculation substitutes the section identities and applies the integral-swap theorem [4].

A Bryony document can retain precisely those checkpoints:

```
let nuS := restrict nu s
let H(omega,t) := indicator (t < Q omega) (w omega *scalar k t)

know H is integrable under product(mu,nuS)
  by product_integrability then measurable_indicator

identify the t-section of H by indicator_section and scalar_integral
identify the omega-section of H by preimage_section and scalar_integral
exchange the two integrals of H by Fubini
conclude the displayed identity using the two section identities
```

The method names here denote proposed registered interfaces, not existing Bryony commands. The section identities remain obligations with exact expressions; no parser is expected to infer them from the English word "section" alone.

## 8.3 The demand graph is more useful than a bag of adjectives

The integrability demand is indexed by both $H$ and $\mu \times \nu_s$. Its route uses an integrability fact for $k$ under $\nu_s$, not under unrestricted $\nu$. The measurability demand refers to the strict subgraph of this exact $Q$. Scalar extraction uses the selected scalar action and compatibility structures.

A useful registry has a small number of reusable consumers: integrability of a product-separable integrand, integrability after a measurable indicator, section evaluation, measure-restriction normalization, and Fubini. It should compose these at the actual objects. Registering a single lemma whose conclusion is exactly (6) is an important baseline, but it would not test the demand engine; it would test theorem lookup.

The companion does not execute these analytical registrations. This case is a detailed integration specification. It is particularly suitable after the arithmetic slice because it tests measure indices, a two-variable relation, and transformations of a joint integrand rather than only scalar algebra.

### 8.4 The non-strict sibling is a semantic test

The second source theorem uses the closed supergraph $Q(\omega) \le t$. It exchanges $[Q(\omega), \infty)$ with $\{\omega : Q(\omega) \le t\}$. The weak inequality is retained on both sides, so mass on the graph is retained. It is not an interchangeable rendering of the strict theorem [4].

To see the danger concretely, take a one-point $\Omega$, $Q = 0$, unit weight and kernel, and let $\nu$ be a unit point mass at zero with $s = \mathbb{R}$. The weak section includes the point mass; the corresponding strict section does not. A rewrite that changes $<$ to $\le$ without a null-boundary argument changes the integral. This is a statement-fidelity failure even if a different, weakened theorem could be proved afterward.

## 9 Observation-sensitive typing and almost-everywhere consumers

### 9.1 Equality is not one universal coercion

A property layer needs relations indexed by their intended observation. Typical examples are pointwise equality, equality on a set, eventual equality in a neighborhood filter, equality almost everywhere under a fixed measure, equality of a finite jet, and a quantitative error bound. None should be implemented as a generic invitation to rewrite an arbitrary consumer.

For a relation $R$ on inputs and an output relation $S$, a consumer $C$ can register a theorem

$$\forall x, y,\ R(x, y) \to S(C(x), C(y)). \tag{7}$$

This is a behavioral contract on the consumer. It tells the elaborator where the observation suffices. When $S$ is equality, the consumer respects the input relation, but the inputs themselves need not be equal.

For example, integration under $\mu$ respects $\mu$-almost-everywhere equality. Point evaluation generally does not. Differentiation at $x$ respects eventual equality near $x$, not agreement only at $x$. Truncating a formal series respects full series equality, whereas reconstructing the entire series from one truncation has no general such theorem.

### 9.2 The almost-everywhere interface

The source already uses `integral_congr_ae` in its final calculation. The corresponding Bryony interface is

$$f =_{\mu\text{-a.e.}} g \quad \Longrightarrow \quad \int f \, d\mu = \int g \, d\mu.$$

It must retain the measure and the target-space structures. Equality under $\mu$ does not license a replacement inside an integral under an unrelated $\nu$. Such a change needs its own measure-comparison theorem.

There is a subtle but useful distinction here: equality of Lean's totalized Bochner integrals under a.e. equality does not itself need an integrability hypothesis. Fubini's analytic interchange, by contrast, uses the integrability of the joint integrand in the source proof. Bryony should register each theorem's actual premises, not add a universal "integrable" guard to every expression containing an integral. Conversely, obtaining an equality between totalized integral expressions should not be advertised as establishing integrability or the intended analytic interchange [4].

A safe display might say "the two integrals are equal by a.e. congruence" and separately list whether integrability is available. This is more precise than a single adjective such as "well behaved."

### 9.3 Quantifiers across null sets

A countable family of a.e. statements can be combined by intersecting countably many full-measure sets. An arbitrary uncountable family cannot generally be combined in that way. Under Lebesgue measure on $\mathbb{R}$, for every fixed $a$ the statement $x \neq a$ holds almost everywhere, but $\forall a \in \mathbb{R},\ x \neq a$ is false for every $x$.

Therefore a rule taking

$$\forall n \in \mathbb{N},\ P(n, x) \text{ for a.e. } x$$

to the a.e. statement $\forall n, P(n, x)$ needs the countability interface. The analogous rule over arbitrary real indices is not admitted. Quantifier order is part of the elaborated proposition and of the source checkpoint; it is not erased when a property is placed in an index.

### 9.4 Precision requirements are another instance of the same idea

Suppose two formal series agree in all coefficients of degree below $N+1$. Their derivatives then agree below degree $N$, because coefficient $n$ of the derivative depends on coefficient $n+1$ of the original. The input demand for derivative precision $N$ is therefore $N+1$. Multiplication, composition, and inversion have their own demand theorems, with guards where needed.

This is not a justification for placing precision arithmetic into unrestricted type conversion. It is an ordinary theorem-backed requirement transformation. A demand can include both a precision condition and a branch or unit condition. Their conjunction is retained, and alternative registered constructions may have different sufficient demands. A successful finite observation remains a finite observation unless a separate coverage theorem establishes the requested global claim.

## 10 Dependent structures: tensor families and witness identity

### 10.1 The source's interface

The selected Fermat theorem takes a finite indexing type $\iota$ and commutative $A$-algebras $H_i$, each finite and free as an $A$-module. It constructs a commutative $A$-algebra $W$, also finite and free, with equivalences

$$e_T : \mathrm{AlgHom}_A(W, T) \simeq \prod_{i:\iota} \mathrm{AlgHom}_A(H_i, T), \tag{8}$$

and a naturality law under postcomposition by $A$-algebra homomorphisms. Its proof proceeds by finite-type induction: the empty case uses $A$, the successor case uses a tensor product, and the equivalence case transports the indexing family [6].

This example cannot be represented faithfully as an unordered collection of independent adjectives. The ring structure on $W$, its $A$-algebra structure, the family $e_T$, and its naturality law refer to one another. A construction result needs an ordered dependent package. Refinements can describe the finite/free properties of the fixed constructed package; they cannot select different algebra structures merely because those make a later proof easier.

### 10.2 A more mathematical surface

A proposed construction block can expose the universal property first:

```
construct W as the finite tensor product of the family H over A
obtain e(T) : AlgHom(A,W,T) <=> forall i, AlgHom(A,H(i),T)
  natural in T
know W is finite and free as an A-module
  from the corresponding properties of every H(i)
```

The finite/free conclusion is a distinct checkpoint from the universal property. Removing the hypotheses on the factors should leave the construction and its universal property available where their own construction theorem permits them, while reopening the stronger module conclusions. It must not make the elaborator silently restore the omitted factor hypotheses.

For a singleton family, a representing algebra with the specified natural universal property is isomorphic to the single factor. Taking $A = \mathbb{Q}$ and $H = \mathbb{Q}[X]$ shows why finite indexing alone cannot justify a finite-dimensional representing object. This is the premise omission corrected by the two syntheses, and the original source includes the needed hypotheses [1, 2, 6].

### 10.3 Why supports cannot identify data

Suppose one route constructs $W_1$ as a nested tensor product and another constructs $W_2$ using a different bracketing. Their universal properties may establish an isomorphism. That is not definitional identity, and an antichain engine must not combine a proof about $W_1$ with a proof about $W_2$ under a single atom "$W$ is finite."

Requirement minimization is performed only after the candidate object and its structures are fixed. Alternative construction candidates have separate demand graphs. A theorem may transport properties along an explicit isomorphism, but that bridge is represented as a theorem application and checked at the actual endpoints.

The same rule applies to program synthesis. Two programs satisfying the same behavioral specification are not the same anchor. Reusing a proof requires equality or a relevant extensional transport theorem, not merely identical result types or overlapping sets of prospective premises.

### 10.4 Existence and computation

The source theorem is an existential statement in Prop. It proves existence of the algebra and its interface. It does not, merely by having that type, expose a canonical executable data object. A constructive package, a noncomputable choice, and a proof of existence are different interfaces. Bryony can support all three, but the source must authorize the chosen one.

Likewise, a proposition-valued finiteness hypothesis is not itself an executable enumeration. A mathematical induction or a classical finite-type construction can use it without pretending that an algorithm has obtained a concrete list of every index. The meaning of a "finite tensor product" interface must specify whether it exports executable data, an existence theorem, or an explicitly noncomputable construction.

## 11 Leant as a producer, not an authority shortcut

### 11.1 Start from the existing stronger baseline

Leant already does more than search a bare arrow type. Its documented workflow re-elaborates candidates in Lean, offers ranked alternatives, and supports model-based Length assessment with carefully delimited filtering behavior. At the accessible snapshot, `Verified candidate` in `Verification.hs` is explicitly an opaque receipt for acceptance by a supplied callback, not itself a

kernel proof or behavioral certificate [7, 8].

`PostVerification.hs` adds a generative epoch to occurrence handles and validates a bounded permutation of the supplied receipts. This prevents omission, duplication, and reassociation at that ordering boundary; it does not prove a new mathematical property of the underlying programs [9]. Bryony should reuse such identity discipline rather than replace it with a loosely associated JSON verdict.

The desired integration adds proof-producing behavioral acceptance at Bryony's exact target. A Leant candidate carries the selected source term, and acceptance retains a proof of the requested $\mathsf{Spec}(f)$ about that term. A callback receipt, a model theorem, an execution log, and a Lean proof are related artifacts with different meanings.

## 11.2 A staged synthesis request

A request contains the fixed carrier, structures, domain guard, behavioral relation, permitted providers, and authorized data holes. A producer may first construct a typed candidate and then discharge its contract. It may also propagate requirements into a typed sketch before completion. Either route ends with the exact candidate and an actual proof at the frozen specification.

For a composition sketch $g \circ f$, Proposition 4.1 produces obligations for the intermediate value. A prospective contract on $f$ is not assumed merely because it would make $g$ applicable. The engine can retain a conditional recipe, search for a proof, or ask the user to supply a missing mathematical premise. It cannot accept the sketch by silently strengthening its inputs.

Proof search can use Leant for genuine structural composition: supplying dependent arguments, assembling an invariant-subtype map, or composing a family of specification theorems. Direct lookup must remain a control. The implementation should report when a single existing theorem solved the obligation rather than claiming that synthesis created a new proof architecture.

## 11.3 Positive abstraction and negative realization

Let $o_A : A \to U$ and $o_B : B \to V$ be observations, and suppose a candidate $f : A \to B$ has a model $\widehat{f} : U \to V$ with a proved commuting equation

$$o_B(f(x)) = \widehat{f}(o_A(x))$$

for admissible inputs. A universal abstract property can be instantiated at the observed value of any admissible input. If that abstract property implies the requested concrete specification, it yields a positive proof. No surjectivity of $o_A$ is required for this direction.

A failing abstract input is different. To refute a concrete universal specification one needs an admissible concrete input realizing the failing observation, together with the transfer implication saying that a true concrete specification would imply the abstract property being violated. Surjectivity of the observation is sufficient, but a realization theorem for this one witness is enough. These directions cannot be collapsed into a single badge of "abstraction correctness" [1, 2].

**Proposition 11.1** (Realized counterexample)**.** *Let $D : A \to \mathsf{Prop}$, let $C(x)$ be the concrete specification of a fixed candidate, and let $M(u)$ be its abstract specification. Suppose*

$$\forall x,\ D(x) \to C(x) \to M(o_A(x)).$$

*If $D(x_0)$, $o_A(x_0) = u_0$, and $\neg M(u_0)$, then $\neg \forall x, D(x) \to C(x)$.*

*Proof.* A claimed universal concrete specification gives $C(x_0)$. The transfer theorem gives $M(o_A(x_0))$, and the realization equality rewrites it to the proposition contradicted by the supplied abstract counterexample. □

### 11.4 Lengths, empty types, and correlated observations

For lists over a type $A$, the image of length is precisely

$$\{n \in \mathbb{N} : \ n = 0 \ \vee \ \text{Nonempty}(A)\}.$$

An empty list realizes zero. A positive-length list supplies an element. Conversely, an element can be replicated to realize any natural length. Thus the constant-empty function on `List Empty` preserves length universally: all its inputs are empty. An abstract failing length of one is not a concrete counterexample.

For a grammar whose list lengths are affine forms

$$c + \sum_{i=1}^{k} a_i n_i,$$

equality of constants and coefficients proves equality at every input. Over `List Nat`, all nonnegative input lengths are realizable, and equality at zero and the basis vectors establishes the converse as well: zero determines the constant, and each basis vector determines its coefficient. Over `List Empty`, only zero lengths are realized, so unrestricted coefficient equality is sufficient but not necessary for source-level equality. The claim must name the domain as well as the grammar.

Joint observations can impose additional constraints. For example, the pair $(|xs|, |xs|)$ has individually unrestricted coordinates over inhabited element types, but its image lies on the diagonal. Combining a first-coordinate witness zero with a second-coordinate witness one does not construct a single input realizing $(0, 1)$. A two-output or multi-observation contract needs a joint realization relation.

These examples explain why a passive provider law or a successful sample bank cannot simply be promoted to a universal proof. The exact provider implementation must be linked to the abstract semantics by a theorem or by a proved interpreter for the allowed candidate grammar. Syntactic polymorphism alone does not supply arbitrary free theorems in Lean with classical choice [12].

### 11.5 Failure states and sketch pruning

Bryony distinguishes unsupported translation, exhausted search, an unresolved conditional proof, a model-relative counterexample, a realized concrete counterexample, and a proof of the negation of the original specification. Only the latter appropriate evidence supports a logical rejection. A surviving candidate after counterexample filtering is not thereby verified behaviorally.

A counterexample to one completed program excludes that program. Excluding a sketch requires a theorem about *every permitted completion*, or else an explicitly heuristic search decision without a completeness claim. A sufficient-condition engine such as Section 5 does not, by itself, provide necessary conditions for all possible programs. It must not be repurposed as a sound universal pruning oracle without an additional theorem.

## 12 Certified computation and a reusable reification boundary

### 12.1 A computation returns a relation

A CAS-style operation should return a value $y$ with evidence for the exact requested relation $R(x, y)$. Polynomial equality, a root witness, a complete solution predicate, an enclosure, and a truncated expansion are distinct requests. A sound root checker need not prove that every root was found; a complete solution interface requires both soundness of every reported solution and coverage of all solutions.

Bryony provides three interchangeable *producer* routes where their guarantees match: a verified algorithm, an untrusted algorithm with a verified certificate checker, and an ordinary proof-producing tactic. None receives permission to alter the target. This follows the computational boundary developed in the earlier rounds, while placing its guards into the requirement calculus [1, 2].

## 12.2 What can be shared among checkers

For a supported expression grammar, an internal reification result can have the form

$$\mathsf{Reified}(a) = \{(e, \rho, p) : e : \mathsf{Syntax},\ \rho : \mathsf{Environment},\ p : [\![e]\!]_\rho = a\}.$$

This is schematic dependent packaging: syntax and environment are data, and the final component is a checked bridge to the original expression. A checker can then reason about $e$ without rediscovering the source correspondence.

Suppose $p_L : [\![e_L]\!]_\rho = L$, $p_R : [\![e_R]\!]_\rho = R$, and a soundness theorem converts an accepted certificate into $[\![e_L]\!]_\rho = [\![e_R]\!]_\rho$. Equality transitivity yields the original target $L = R$. Changing $L$, the interpretation, or an opaque atom invalidates that bridge even if the certificate still checks against its own syntax.

A common arithmetic grammar can serve several checkers, but only where its semantics match. Natural-number length arithmetic is a semiring fragment; integer polynomial certificates use ring operations; field normalization has nonzero-denominator guards. Sharing a syntax datatype does not prove that these interpretations agree. A reasonable first library factors addition and multiplication through a proved semiring core and introduces subtraction, rational coefficients, and division through separate typed extensions.

## 12.3 A representative ideal certificate

For polynomials $p, g_1, \ldots, g_m$ over a commutative ring, a certificate giving polynomials $q_i$ with

$$p = \sum_{i=1}^{m} q_i g_i$$

proves that $p(v) = 0$ whenever every $g_i(v) = 0$. Evaluation preserves sums and products, so the conclusion follows by substituting the zero premises. The checker must prove the polynomial identity in its data representation and connect that representation to polynomial evaluation.

The certificate demonstrates membership in the generated ideal; failure to discover one does not establish nonmembership. Coercing the coefficient ring can change which certificates exist. For instance, $X$ belongs to the ideal generated by $2X$ over $\mathbb{Q}[X]$, but not over $\mathbb{Z}[X]$. A computation over the larger ring cannot silently answer the original membership request over the smaller one.

## 12.4 Execution is a separate proof obligation

A theorem saying that `check c = true` implies the target is not evidence that an external process actually computed that equality. The strict route proves the checker result by kernel reduction or produces a proof by reconstruction. An accelerated execution route has a distinct, toolchain-specific trust policy. A digest records identity; it does not prove arithmetic.

Bryony's first implementation should reuse existing Lean arithmetic tactics and the already small checked components reported in the syntheses. A larger reflective checker is justified by a workload and a measured cost, not by a desire to call the whole system a CAS. The property engine, certificate producer, checker, and original-target bridge remain separable, so each can be tested or replaced without changing the others' guarantees.

# 13 Extending Lean's type system

## 13.1 Author-facing refinement checking

Lean already represents subtypes and proofs of arbitrary predicates; its subtype documentation explicitly separates an underlying value from its propositional property [11]. Bryony first extends the checking discipline around those representations. At a use site, checking $e$ against $A$ where $P$ checks $e : A$ and generates the demand $P(e)$. A property-implicit binder discharges that demand or leaves a visible conditional template.

The useful extension is therefore not greater logical expressiveness. It is a systematic mechanism for propagating specifications through composition, reconstructing proof arguments, and explaining incomplete applications. The inference process can be proof-relevant even when the resulting property proofs are irrelevant for runtime computation. Runtime erasure does not imply zero elaboration cost, zero proof size, or zero checking cost.

## 13.2 Law-bearing and behavior-indexed arrows

A second extension makes behavioral and relational specifications first-class in signatures. A guarded arrow can be defined by

$$(A \xrightarrow[P]{Q} B) := \{f : A \to B \mid \forall x,\ P(x) \to Q(x, f(x))\}.$$

A richer signature permits a whole-function predicate $L(f)$ in addition to the pointwise contract. An inverse operation can return a pair $(f, g)$ with both inverse laws; a functorial construction can return its action on objects and morphisms with the relevant laws. These are dependent records rather than a new family of trusted logical axioms.

Bidirectional checking is natural here. An expected contract can propagate demands backward into an expression; an inferred contract can supply consequences forward. A specification-preserving coercion is an explicit theorem application. The variance theorem in Section 4 gives the correct rule for guarded arrows, while genuinely dependent result types may require transport at additional indices. A simple covariance annotation on the carrier arrow is not sufficient to derive all those transports.

This layer should coexist with effectful program verification rather than replace it. A pure mathematical predicate on a function is different from a Hoare specification for state, exceptions, or I/O. Lean's existing `mvcgen` infrastructure is a relevant baseline for effects; Mathlib's `fun_prop` already performs compositional reasoning about registered function properties [14, 13]. Bryony's experiment is the cross-cutting authoring interface and retained requirements, not the invention of compositional property automation.

## 13.3 A native refinement former, precisely stated

A possible kernel experiment adds a primitive

$$\mathsf{Ref}(A, P), \qquad A : \mathsf{Type}_u, \quad P : A \to \mathsf{Prop},$$

in $\mathsf{Type}_u$, with constructor and projections

$$\frac{a : A \quad p : P(a)}{\mathsf{pack}(a, p) : \mathsf{Ref}(A, P)}, \quad \mathsf{val}(r) : A, \quad \mathsf{evidence}(r) : P(\mathsf{val}(r)).$$

The computational rules would include

$$\mathsf{val}(\mathsf{pack}(a, p)) \longmapsto a, \qquad \mathsf{evidence}(\mathsf{pack}(a, p)) \longmapsto p.$$

Subsumption still needs a proof:

$$h : \forall a,\ P(a) \to Q(a) \quad \Longrightarrow \quad \mathsf{weaken}_h(r) = \mathsf{pack}(\mathsf{val}(r), h(\mathsf{val}(r))(\mathsf{evidence}(r))).$$

There is no rule saying that an arbitrary $a : A$ automatically inhabits $\mathsf{Ref}(A, P)$ because the elaborator hopes to prove $P(a)$ later.

The natural interpretation is

$$[\![\mathsf{Ref}(A, P)]\!] = \{a : [\![A]\!] \mid [\![P]\!](a)\},$$

with constructor and projections interpreted by ordinary subtype operations. If the native syntax adds exactly these definitional expansions, it adds no expressive power over the encoding. Its possible benefits would be implementation-level: compact proof-field storage, faster transport, clearer errors, or simpler internal representations.

A native implementation has a larger obligation than a macro expansion. It must preserve substitution, universes, typing, and the relevant reduction behavior, and its conversion rules must translate to conversions or explicit proofs accepted by the destination theory. Any new judgmental equality—for example a proposed definitional functoriality law for repeated refinement transport—requires a separate argument. One cannot infer its conservativity merely because the types themselves resemble subtypes.

### 13.4 What should not become definitional equality

The property implication $P(a) \Rightarrow Q(a)$ is theorem proving, not ordinary reduction. Putting arbitrary semantic subsumption into conversion would couple type checking to open-ended entailment search. The correct outcome of a bounded search is then unknown, not a conversion success or a proof that the types are different. A CAS that establishes a propositional polynomial identity can supply an equality proof; it need not and should not silently enlarge conversion for every later elaboration.

Nor does proof irrelevance identify distinct ring multiplications, distinct measures, or different choices of an inverse. It identifies proofs of the same proposition in Lean's logic, not arbitrary data satisfying similar laws. The user-facing type system must retain this distinction even if the kernel offers optimized proof storage [10, 12].

The first kernel experiment should therefore be conditional on a measured bottleneck in the best ordinary-Lean implementation. Record the cost attributable to subtype wrappers, transport terms, repeated checking, and evidence storage. Attempt the smallest primitive that addresses that cost, then compare its translated and native versions under the same semantics. A speculative native feature should not delay a property-aware binder that can already be implemented by metaprogramming.

## 14 Rewriting, scopes, and proof-carrying repair

### 14.1 An anchor is an elaborated expression

A property about $t$ is keyed by its actual expression and dependent context, including implicit structures. If simplification replaces $t$ by $u$, a proof of $P(t)$ can be reused directly when the expressions are definitionally equal under the checked context. Otherwise, an equality $t = u$ supplies a transport to $P(u)$. A theorem $P(t) \leftrightarrow Q(u)$ instead supplies a proposition-level adapter, without asserting equality of the underlying objects.

The old fact need not disappear when the goal is simplified. The index can retain the old anchor and apply an explicit bridge when a consumer asks for the rewritten property. Normalized strings

and pretty-printed names can help locate candidates, but are not transport evidence. Dependent indices and structure arguments remain in the elaborated target presented to Lean.

A practical first implementation should support a narrow set of transport forms: conversion, homogeneous equality, explicit equivalence-of-propositions, and registered observation consumers. Other cases remain obligations. This is preferable to an ambitious transport layer that silently identifies objects with similar displays.

## 14.2 Context embeddings instead of persistent local names

An old local context

$$x_1 : A_1, \ldots, x_n : A_n(x_1, \ldots, x_{n-1})$$

is reusable in a new context only through a checked substitution assigning each old variable a term of its instantiated type and each needed hypothesis a proof. Local identifiers surviving a rename or a file edit do not supply that substitution.

For the first release, the evidence index should have the lifetime of one elaboration snapshot. Rebuild it after a declaration is rebuilt. Persist closed proof artifacts and source associations, but replay them against the new context. Add incremental migration only after measurement shows that rebuilding is a significant cost and the intended transport shapes are clear.

Sibling branches are separate contexts. A nonzero fact from one branch cannot discharge a demand in the other. To join branches, the resulting theorem uses case elimination. To export a fact depending on an introduced witness, retain the existential or dependent package that binds it. An unordered support set is only the nondependent projection of this structure.

## 14.3 Repair means replaying a theorem, not restoring a label

Suppose a proof of the cast identity used the support $\{2 \mid b, 4 \leq p\}$. After an edit removes evenness, that route becomes conditional. Another route may still prove the same target from direct divisibility. The editor can propose it and retain its proof. If no route closes, the checkpoint remains open.

If the edit changes $b$ itself, a proof about the old $b$ is not reused just because the property name is unchanged. If the user authorizes a new synthesized function, it receives a new candidate identity and a new proof of its specification. A historical proof is historical evidence, not a transferable certificate for an altered program.

Alternative supports also allow useful explanations of fragility. A statement with several independently provable support routes may survive an edit that breaks one. This is not a formal guarantee of future robustness: the rule pool and all its theorem dependencies can change. It is a current account of known proof alternatives.

## 14.4 Truth, statement fidelity, and method fidelity

The acceptance boundary has three questions. Does the retained term prove its formal target under the selected axiom policy? Does that target correspond to the author's statement, including its hidden arguments? Does the derivation respect an explicitly required method or allowed support?

A contradiction shortcut in a full Frey-package context can pass the first two while failing an arithmetic-method experiment. A proof of an enclosure instead of equality fails the second even if the enclosure is true. A theorem name mentioned in an unused subterm does not establish method fidelity.

A required reason should therefore select an explicit rule profile or a small derivation interface, not merely require textual occurrence of a theorem constant. A search hint is different: it guides

search but does not constrain the accepted argument. The document should make the mode clear. Whole-proof inspection can supplement this policy, but no system should claim to recover a mathematician's intention from arbitrary proof-term syntax alone.

# 15 The executable reference model

## 15.1 What runs

The archive contains `prototype/bryony.py`, a Python 3.9+-compatible implementation using the standard library. It parses a small ASCII language of declared proposition names, known facts, prospective premises, named ground rules, and one goal. It computes antichain labels, emits conditional proof trees, independently replays those trees, and exports ordinary Lean theorem templates for the propositional derivations.

This is an actual executed slice of the requirement algorithm, not a parser for the proposed mathematical surface. Its rule declarations are assumptions of a finite propositional model. The model does not verify the mathematical meaning of an atom such as `NumeratorExact`, elaborate dependent Lean expressions, register Mathlib theorems, or invoke Leant. Its exported Lean statements quantify explicitly over proposition variables and rule assumptions. They contain proof terms, but were not compiled here because this preparation environment did not contain a Lean toolchain.

## 15.2 Independent replay and identity checks

A certificate contains the exact source snapshot, its digest, the target, the claimed support, and a tree of known leaves, prospective leaves, and rule applications. The replay checker does not call the search procedure. It checks node forms, conclusion and premise matching, arity, licensed leaves, and the exact union of prospective dependencies.

At this model boundary, replay compares the full source snapshot, not only a hash. A changed rule set, branch label, candidate identity, or policy label invalidates an old certificate. This tests exact-snapshot behavior; it is not an implementation of checked Lean context embeddings. The digest is retained for diagnostics and reproducibility, not as a substitute for a semantic proof.

The reference engine stores immutable proof trees and retains only the first proof for an equal support. It can therefore lose a cheaper or method-preferred proof having the same premises. A production system should either filter its method policy before closure, as assumed here, or maintain a separate policy/cost-indexed family. Support inclusion alone is not a valid dominance criterion across different policy guarantees.

## 15.3 Executed results

The complete run is recorded in `evidence/results.json` and `evidence/test.log`. The results are:

| Check | Result |
|---|---|
| Unit-test methods, including the differential and arithmetic groups | 23 passed |
| Deterministically generated finite rule systems | 400 |
| Exhaustively checked prospective-premise subsets across those systems | 3,873 |
| Atom-level comparisons with the independent closure oracle | 2,800 |
| Generated random-case proof certificates independently replayed | 1,835 |
| Positive Frey arithmetic triples checked exactly | 288 |
| Single-premise arithmetic mutations checked | 4 |

The random generator uses seed 4406. Each system has seven proposition names, up to five prospective premises, and up to twelve rules with zero to three premises; repetitions and cycles are permitted. For every prospective-premise subset, an independent oracle computes ordinary forward fact closure without antichain optimization. Its minimal supports are compared with the engine's supports for every atom. Every generated retained proof is replayed separately.

The adversarial tests include missing seeds in cycles, seeded cycles, zero-premise rules, dominated supports, duplicate premises, forged supports, altered goals, wrong rule arities, unlicensed leaves, changed snapshots, changed candidates, cyclic certificate objects, and resource limits. A tiny observation-consumer fixture permits a.e. integral congruence while leaving point evaluation unresolved; this tests registry behavior, not measure theory.

These numbers are not independent formal theorems, and they do not measure the cost of a Lean evidence index. The run is small enough that its elapsed time is primarily a reproducibility detail. The mathematical justification of the algorithm is Theorems 5.1 and 5.2; mechanizing that metatheory and connecting registered rule instances to actual Lean proofs are distinct next tasks.

### 15.4 Reproduction

From the archive root:

```
python prototype/test_bryony.py
python prototype/bryony.py prototype/frey4.bry --output evidence/frey4
latexmk -pdf -interaction=nonstopmode -halt-on-error Bryony.tex
```

The generated `evidence/frey4/Requirements.lean` is a prospective replay artifact, not a log of successful Lean compilation. On a selected Lean installation, `lean` applied to `evidence/frey4/Requirements.lean` is the direct next check. The abstract rules remain explicit hypotheses even after that file compiles; compiling it would not turn the names in the Python fixture into proved arithmetic registrations.

## 16 Implementation and evaluation that can change the decision

### 16.1 A minimum Lean package

The first package should contain a property-implicit application mechanism, a snapshot-local fact index, a small theorem registry, bounded ground-instance generation, proof-carrying labels, and an obligation renderer. The first arithmetic workload is the exact-quotient use site, reusing the helper theorems reported by the syntheses rather than reproving them under new names.

The second workload is the weighted Fubini construction. It must reuse the same identity and requirement mechanisms while adding measure-sensitive consumers and section identities. Because this article has already inspected the file, it is a development transfer case, not a held-out task. A genuine held-out task must be chosen independently after the registry and evaluation protocol are frozen.

Leant integration should begin with one real candidate-producing adapter and a fixed behavioral grammar whose source correspondence is checked. A separate native refinement kernel should not be a prerequisite. The implementation sequence is determined by these proof and integration gates, not by calendar estimates.

## 16.2 Keep the controls equally equipped

Use the following comparison arms, following the syntheses but preserving the allocation precisely [1, 2]:

| Arm | Available facilities |
|---|---|
| L0 | Ordinary Lean, including applicable existing automation. |
| L1 | L0 plus the new mathematical packages and theorem registrations. |
| L2 | L1 plus the same added synthesis and certificate services. |
| L3 | L2 plus Bryony's requirements, property-implicit arguments, and diagnostics. |
| L4 | L3 plus a separate mathematical input or document syntax, if implemented. |
| LR | L2 with a read-only mathematical rendering of accepted proofs. |

The property-layer question is L3 versus L2, not L4 versus unassisted Lean. The LR arm isolates comprehension gains due to display. Existing `fun_prop`, theorem lookup, simplification, arithmetic tactics, and appropriate program-verification support belong in the baseline where applicable. New mathematical packaging must not be withheld from the controls.

## 16.3 Measure authoring and implementation costs separately

For authoring, record active time, explicit routine proof interventions, diagnosis time, repair effort, and semantic-reading errors. A reader should recover the actual quantifiers, objects, measures, domains, precision, and conclusion strength from the compact view before seeing its expansion. A prettier statement that invites a stronger interpretation is a failure.

For the implementation, record index entries and bytes, rule instances generated, rejected instances, rule firings, support-family sizes, proof-DAG size, proof transport size, kernel-checking time, and rebuild cost per declaration. Separate single-lemma lookup from multi-step composition. Report registration and maintenance effort, including the infrastructure supplied by the research team.

There are no real-file measurements of those quantities in this article. The Python insertions and test timings are not substitutes. After a pilot estimates variability, predeclare thresholds for the main comparison. A negative outcome is useful: retain the libraries or renderer and remove an interface whose gains disappear against the controls.

## 16.4 Paired acceptance tests

Every integration test needs a positive neighbor and a stated failure kind. Important pairs include exact versus inexact quotient transport; strict versus non-strict fibers with and without a null-boundary bridge; a.e. integral congruence versus unlicensed point evaluation; a fixed structure versus a changed implicit structure; a rebuilt context versus stale local identifiers; and finite/free tensor factors versus finite indexing alone.

For behavioral acceptance, include realizable and unrealizable abstract counterexamples, correlated observations, and one failed completion versus all completions of a sketch. For document publication, include an unused unsupported assertion. For required-method evaluation, supply a satisfiable helper context and a disallowed shortcut in a larger contradictory context.

A rejected proof attempt, an unsupported adapter, an open requirement, and a proved negation are distinct expected outcomes. Tests should not reward an implementation for rejecting a valid theorem merely because one convenient sufficient route is unavailable.

# 17 Conclusion

Bryony treats a mathematical proof as a sequence of meaningful constructions and claims connected by theorem-backed adapters. Its type layer says not only what carrier an object inhabits, but which properties have actually been proved, which relations an operation preserves, and which premises a proposed use still needs.

The central design choice is to retain *alternative conditional proof recipes*. This makes inference explainable without pretending that the engine knows the weakest possible hypotheses. It also unifies several practical tasks: filling property-implicit arguments, composing behavioral contracts, tracking analytical guards, and repairing a proof after an assumption changes. The finite algorithm has a clear scope, a proof of relative completeness, and an executed reference model.

The larger language remains a proposal. The hard integration work is preserving actual Lean expressions, dependent contexts, and theorem identities while making that mechanism useful to an author. Native refinement types are a legitimate later experiment, but none of the worked examples needs a new logical axiom or unrestricted semantic conversion. The next decisive artifact is a small Lean implementation of the exact-quotient and analytical use sites, compared against the same libraries and services without Bryony's interface.

# A Responses to the two round-three question sets

The following tables answer the implementation questions posed by the two syntheses. An unanswered empirical quantity is identified as such rather than replaced by a design assertion.

## A.1 Questions from *From Properties to Proof Obligations*

| Question | Bryony's decision or evidence |
|---|---|
| First use-site interface? | Property-implicit theorem application and a requirement view; fixed Frey exact-quotient use site first. Separate narrative parsing is not required (Sections 3, 7). |
| Finite automatic fragment? | Finite prospective premises and grounded Horn rules over fixed elaborated terms. Theorems 5.1–5.2 apply after grounding; generation has separate limits. |
| What may inference decide about data? | Only explicitly authorized witness holes. Meaning-bearing target parameters and structure choices are sealed before provider search (Section 6). |
| Required methods? | A selected rule profile or a checked derivation interface, distinct from a search hint. Token occurrence does not establish method fidelity (Section 14). |
| First behavioral bridge? | A fixed list grammar with a checked source/interpreter correspondence and its length model. Positive soundness and any converse must be proved separately; no such integrated bridge is newly claimed here (Section 11). |
| Negative Leant selection? | Require a proof of the original negation or a realized admissible witness with negative transfer. Preserve model-relative failures as a weaker status. |
| First nonalgebraic transfer? | Weighted subgraph Fubini is the inspected development transfer. Select a genuinely unseen analytical task after the profile is frozen. |
| Retained evidence? | Exact target, context, structures, candidate, proof or replayable certificate, theorem dependencies, source map, and policy. Rebuild local indices initially (Section 14). |
| When enlarge the checker? | After measuring the original-target bridge and kernel checking against existing tactics. No native-execution crossover is claimed (Section 12). |
| When stop the language experiment? | When L3 adds no worthwhile authoring or repair benefit over L2, or worsens reading accuracy or maintenance cost. Retain useful packages and rendering independently. |

## A.2 Questions from *Nine Refinements*

| Question | Bryony's decision or evidence |
|---|---|
| T1. Real-file evidence-index cost? | Not measured. The companion reports finite model counts only. Per-declaration Lean telemetry is a first integration requirement. |
| T2. Which registry rules fire? | The model's three-rule arithmetic example records eight combinations across two scans. Actual Mathlib registration frequencies and lookup/composition attribution remain to be measured. |
| T3. Does simplification break anchors? | Preserve the old anchor and use checked conversion, equality transport, or a proposition adapter. Never match only normalized display strings (Section 14). |
| T4. Shared reification? | Share a typed syntax/interpreter layer and original-expression bridges; separate semiring, ring, field, and behavioral interpretations and their guards (Section 12). |

| Question | Bryony's decision or evidence |
|---|---|
| T5. Completeness for Leant abstractions? | State it per grammar, admissible domain, observation, and specification. The affine criterion is complete over realizable unrestricted lengths, not over an empty element type without restricting the domain. |
| T6. Almost-everywhere consumers? | Register integral congruence at the exact measure, not a universal equality coercion. Countable quantifier combination is separate from uncountable families (Section 9). |
| T7. Lexical structures and FLT preambles? | Keep lexical structure selection for meaning stability. No shortening measurement is supplied, and generated environment reduction is not credited as mathematical inference. |
| T8. Bounded rule generation? | Fixed term pool, role-checked theorem schemas, bounded supported matching, and no recursive fresh-term growth inside an epoch (Section 6). |
| T9. What does a reader recover? | Not experimentally measured. The evaluation asks for reconstruction of domain, quantifiers, measure, precision, and guarantee before expansion. |
| T10. One implementation enough? | One reference algorithm suffices to fix the finite semantics; an independent replay checker and differential oracle already expose disagreements within that model. Independent Lean integrations against the same paired tests would test the larger specification. |

## B Archive map and limitations

| File or directory | Contents |
|---|---|
| `Bryony.tex`, `Bryony.pdf` | This self-contained article and its rendered PDF. |
| `prototype/bryony.py` | Parser, requirement solver, independent certificate replay, and generic Lean export. |
| `prototype/test_bryony.py` | Differential, arithmetic, identity, and mutation tests. |
| `prototype/frey4.bry` | The finite exact-quotient requirement fixture. |
| `evidence/results.json`, `evidence/test.log` | The executed model results and complete test output. |
| `evidence/frey4/` | Generated requirement certificates and uncompiled Lean templates. |
| `sources.json`, `README.md` | Source pins, scope notes, build and reproduction instructions. |

The mathematical source repositories are read-only inputs; the archive does not contain copies of their full articles or codebases. The model is a small demonstrator, not a production parser or a hardened certificate-processing service. It uses proof trees rather than globally hash-consed DAGs and has ordinary Python resource limits. Unsupported large or deeply nested certificates should fail rather than acquire a truth status. The proposed Lean integration requires additional work on registration, target correspondence, context transport, proof retention, and tooling.

## References


[1] Codex. *From Properties to Proof Obligations*. Round-three synthesis, revised 6 September 2026. Brainstorm, commit `bf33339059950608 70f8585e297ffc16f3fe7e86`. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf333905995060870f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex.

[2] Claude (Anthropic). *Nine Refinements*. Round-three synthesis and reconciliation, 6 September 2026. Same Brainstorm commit. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex.

[3] Vladimir Reshetnikov et al. *ProveIt*. Inspected revision `24ce8bd743eaab64a91ce90725ea00f498d319d2`; `lean-toolchain` pins Lean 4.32.0. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/lean-toolchain.

[4] ProveIt contributors. *Weighted Fubini transport across measurable subgraphs and supergraphs*. `SubgraphFubini.lean`, both theorem statements and proofs, revision above. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/SubgraphFubini.lean.

[5] Fermat repository contributors. `S_FreyPackage_freyCurveInt_map.lean`. Inspected revision `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`; coefficient-map proof and surrounding arithmetic interfaces. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean.

[6] Fermat repository contributors. `S_Algebra_exists_algHom_equiv_pi.lean`. Same inspected revision; finite/free factor assumptions, representing algebra, and naturality. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean.

[7] Vladimir Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. Public revision resolved during this review: `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. README and documented synthesis/Length interfaces. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/README.md.

[8] Leant contributors. `src/Leant/Synth/Verification.hs`. Same public revision; callback receipts and exact-spelling candidate verification. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs.

[9] Leant contributors. `src/Leant/Synth/PostVerification.hs`. Same public revision; epoch-scoped occurrence handles and validated post-verification ordering. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/PostVerification.hs.

[10] Lean team. *The Lean Language Reference: The Type System*. Official documentation consulted 6 September 2026. Live documentation, not a claim of identical behavior at every repository pin. https://lean-lang.org/doc/reference/latest/The-Type-System/.

[11] Lean team. *The Lean Language Reference: Subtypes*. Official documentation consulted 6 September 2026.
https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/.

[12] Lean team. *The Lean Language Reference: Axioms*. In particular the discussion of classical choice and unrestricted parametricity. Consulted 6 September 2026.
https://lean-lang.org/doc/reference/latest/Axioms/.

[13] Mathlib contributors. *Mathlib.Tactic.FunProp*. Official module documentation, consulted 6 September 2026.
https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.

[14] Lean team. *The Lean Language Reference: The mvcgen tactic, Overview*. Official documentation, consulted 6 September 2026. The syntheses' version-specific observations concern their Lean 4.32.0 pin.
https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/.

[15] Johan de Kleer. Problem Solving with the ATMS. *Artificial Intelligence* **28** (1986), 197–224. Primary author-hosted version; supporting environments and justifications, especially pp. 198–200.
https://dekleer.org/Publications/Problem%20Solving%20with%20the%20ATMS.pdf.

[16] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid Types. *PLDI* (2008), 159–169. DOI: 10.1145/1375581.1375602. Primary author page:
https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.

[17] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance Semirings. *PODS* (2007), 31–40. DOI: 10.1145/1265530.1265535.
https://doi.org/10.1145/1265530.1265535.